

## DevOps End-to-End Hands-On Task

### Project Title

Automated Cloud-Native Application Deployment on AWS

### Objective

Students will design, build, automate, and deploy a containerized web application on AWS using industry-standard DevOps tools. The goal is to simulate a real production DevOps workflow covering infrastructure provisioning, configuration management, CI/CD, containerization, orchestration, and automation.

### Technologies to Be Used

- Linux
- Bash
- AWS (EC2, Load Balancer, Auto Scaling Group, S3, VPC, Lambda)
- GitHub
- Docker
- Kubernetes (EKS)
- Jenkins
- Terraform
- Ansible

### Scenario

You are a DevOps Engineer for an organization deploying a web application. The application must be:

- Highly available
- Scalable
- Automated from code commit to deployment
- Secure and production-ready

### Task Breakdown

#### Phase 1: Linux & Bash (Server Preparation)

##### Tasks:

- Launch an Amazon EC2 (Ubuntu) instance.
- Create a non-root user.
- Configure SSH access.
- Install Docker, Git, Java, and Kubernetes CLI tools.

- Write a Bash script to install Docker, start and enable the Docker service, and add the user to the Docker group.

**Deliverables:**

- Bash script (setup.sh)
- Output logs showing successful installation

## Phase 2: GitHub (Source Code Management)

**Tasks:**

- Create a GitHub repository.
- Push a sample application (Node.js / Python / Java).
- Maintain main and dev branches.
- Use meaningful commit messages.
- Add a README.md explaining the project.

**Deliverables:**

- GitHub repository link
- Commit history

## Phase 3: Docker (Containerization)

**Tasks:**

- Write a Dockerfile.
- Build the Docker image locally.
- Test using docker run.
- Tag the image properly.

**Deliverables:**

- Dockerfile
- Docker image build output
- Running container screenshot/log

## Phase 4: Terraform (Infrastructure as Code)

**Tasks:**

- Provision a VPC with public and private subnets and an Internet Gateway.
- Create a Jenkins EC2 instance.
- Create an Application Load Balancer (ALB).
- Create an Auto Scaling Group (ASG).
- Create an S3 bucket for Terraform state or artifacts.
- Create IAM roles and security groups.

**Deliverables:**

- main.tf
- variables.tf
- outputs.tf
- terraform apply output

**Phase 5: Ansible (Configuration Management)****Tasks:**

- Configure EC2 instances.
- Install Docker.
- Install Kubernetes tools.
- Configure the application runtime.
- Use an inventory and playbooks.

**Deliverables:**

- Ansible playbook (site.yml)
- Inventory file
- Playbook execution output

**Phase 6: Jenkins (CI/CD Pipeline)****Tasks:**

- Install Jenkins on EC2.
- Configure GitHub webhook and credentials.
- Create a pipeline to pull code, build a Docker image, push it to Amazon ECR or Docker Hub, and deploy to Kubernetes.

**Deliverables:**

- Jenkinsfile
- Pipeline execution screenshots
- Jenkins job configuration

**Phase 7: Kubernetes (EKS)****Tasks:**

- Create an Amazon EKS cluster using Terraform.
- Deploy the application using Deployment, Service, and Ingress (ALB Ingress Controller).
- Verify pods are running.
- Verify the application is accessible through the Load Balancer.

**Deliverables:**

- Kubernetes YAML files
- kubectl get pods output
- Application URL

## Phase 8: AWS Lambda (Automation Task)

### Tasks:

- Create a Lambda function to stop EC2 instances during non-business hours or clean unused Docker images in Amazon ECR.
- Trigger the Lambda function using Amazon EventBridge.

### Deliverables:

- Lambda code
- Trigger configuration screenshot
- Execution logs

### Final Validation

- Application accessible via Load Balancer
- CI/CD pipeline working end-to-end
- Infrastructure created using Terraform
- Configuration handled via Ansible
- Containers running in EKS